

Laporan M2 Tugas Besar 1

IF2010 Pemrograman Berorientasi Objek 2025/2026

Nimonspoli

Kode Kelompok : STD

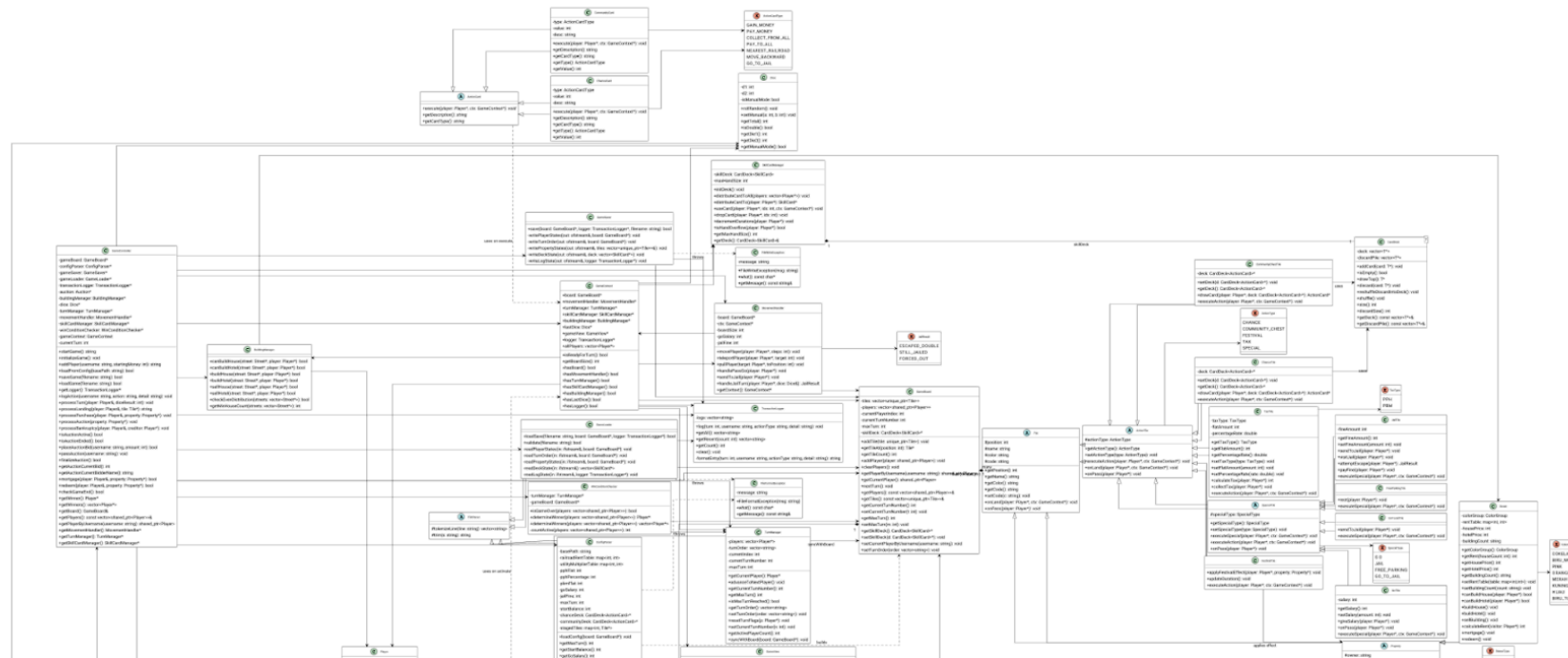
Nama Kelompok : SidTheDestroyer

1. 13524114 / Mirza Tsabita Wafa'ana
2. 13524115 / Ega Luthfi Raiz
3. 13524121 / Billy Ontoseno Irawan
4. 13524125 / Muhammad Rafi Akbar
5. 13524129 / Eduard Daniel Aria Jaya
6. 13524141 / Ahmad Fauzan Putra



STD - SidTheDestroyer

1. Diagram Kelas



Gambar 1.1 Diagram Kelas
Untuk melihat full image: [📄 OOP diagram](#)

2. Deskripsi Kelas

2.1. Kelas CommandProcessor

Kelas `CommandProcessor` berfungsi membaca perintah yang dimasukkan melalui CLI dan menerjemahkannya menjadi aksi yang sesuai dalam sistem permainan.

Atribut:

- `gameController`: digunakan untuk memanggil logika utama permainan
- `boardView`, `propertyView`, `gameView`: digunakan untuk menampilkan output

Method:

- `readCommand()`: membaca input dari pengguna
- `executeCommand(cmd)`: mengeksekusi perintah sesuai input

Relasi:

- Berinteraksi langsung dengan `GameController` untuk menjalankan aksi
- Menggunakan kelas view (`BoardView`, `PropertyView`, `GameView`, `PlayerView`) untuk menampilkan hasil

2.2. Kelas `BoardView`

Kelas `BoardView` berfungsi untuk menampilkan papan permainan ke layar sesuai dengan kondisi terbaru permainan.

Method:

- `showBoard(board, players)`: menampilkan seluruh isi papan termasuk posisi pemain, kepemilikan properti, dan status lainnya

Relasi:

- Mengakses `GameBoard` untuk mendapatkan informasi petak
- Mengakses `Player` untuk mengetahui posisi pemain

2.3. Kelas PropertyView

Kelas PropertyView digunakan untuk menampilkan informasi terkait properti seperti status kepemilikan dan nilai properti dalam permainan.

Method:

- showPropertyDetail(property): menampilkan detail akta kepemilikan suatu properti
- showPlayerProperties(player): menampilkan daftar properti milik pemain

Relasi:

- Menggunakan data dari Property
- Menggunakan data dari Player

2.4. Kelas GameView

Kelas GameView berguna untuk menampilkan pesan umum kepada pengguna.

Method:

- showMessage(msg): menampilkan pesan biasa
- showError(msg): menampilkan pesan kesalahan

2.5. Kelas PlayerView

Kelas PlayerView digunakan untuk menampilkan informasi status pemain.

Method:

- showPlayerStatus(player): menampilkan informasi seperti uang, posisi, dan status pemain

Relasi:

- Mengakses data dari Player

2.6. Kelas Tile

Kelas Tile merepresentasikan petak yang dapat didatangi pemain pada papan permainan. Semua jenis tile (property, action, special, dll.) merupakan turunan dari kelas ini.

Method:

- onLand(player): efek yang diaplikasikan pada player ketika mendarat di petak
- onPass(player): efek yang diaplikasikan pada player ketika melewati petak

2.7. Kelas Property extends Tile

Kelas Property adalah turunan dari Tile yang merepresentasikan petak yang bisa dimiliki pemain. Kelas ini juga bertanggung jawab untuk logika ekonomi dasar seperti menghitung sewa, menyimpan nilai beli dan mortgage, serta mengelola kepemilikan dan status properti.

Method:

- calculateRent(player): menghitung sewa yang dikenakan pada player (selain pemain yang memiliki) yang mendarat di petak properti
- mortgage(): mengubah status menjadi MORTGAGED
- redeem(): mengembalikan status menjadi OWNED
- setFestivalMultiplier(multiplier): menentukan pengali biaya sewa ketika petak diberi efek festival
- setFestivalDuration(duration): menentukan durasi berlangsungnya efek festival pada petak

2.8. Kelas Street extends Property

Kelas Street merepresentasikan properti tipe jalan yang dapat dibangun rumah/hotel. Kelas ini mengelola pembangunan rumah dan hotel, mengatur harga bangunan, menentukan sewa berdasarkan jumlah bangunan, dan mengelompokkan properti berdasarkan warna.

Method:

- canBuildHouse(player): mengecek apakah pemain boleh membangun rumah
- canBuildHotel(player): mengecek apakah pemain boleh membangun hotel
- buildHouse(): membangun rumah pada petak
- buildHotel(): membangun hotel pada petak
- sellBuilding(): menjual bangunan yang ada pada petak

2.9. Kelas Railroad extends Property

Kelas Railroad berfungsi untuk merepresentasikan properti jenis stasiun dalam permainan. Kelas ini menangani perhitungan biaya sewa berdasarkan jumlah Railroad yang dimiliki oleh pemain.

Method:

- getRent(railroadCount): menghitung dan mengembalikan sewa sesuai jumlah Railroad yang dimiliki pemain.

Relasi:

- Inheritance dari Property
- Menggunakan tabel sewa rentTable

2.10. Kelas Utility extends Property

Kelas Utility digunakan untuk merepresentasikan properti jenis utilitas dalam permainan. Memperhitungkan biaya sewa berdasarkan hasil dadu dan jumlah Utility yang dimiliki pemain.

Method:

- `getRent(diceTotal, utilityCount)`: menghitung dan mengembalikan nilai sewa berdasarkan hasil dadu dan jumlah Utility yang dimiliki pemain

Relasi:

- Menggunakan `multiplierTable` untuk menentukan faktor pengali sewa
- Menggunakan hasil dadu dan jumlah Utility pemain

2.11. Kelas `ActionTile` extends `Tile`

Kelas `ActionTile` berguna untuk merepresentasikan petak aksi dalam permainan. Kelas ini menangani efek atau aksi yang terjadi saat pemain mendarat di petak tersebut.

Method :

- `executeAction(player)`: menjalankan aksi sesuai jenis petak terhadap pemain yang mendarat

Relasi:

- Menggunakan `ActionType`
- Berinteraksi dengan `Player`

2.12. Kelas `ChanceTile` extends `ActionTile`

Kelas `ChanceTile` berfungsi untuk memberikan kartu kesempatan kepada pemain saat mendarat.

Method:

- drawCard(player): mengambil dan mengembalikan kartu dari cardPool untuk pemain

Relasi:

- Menggunakan List<Card> sebagai kumpulan kartu
- Berinteraksi dengan Player

2.13. Kelas CommunityChestTile extends ActionTile

Kelas ChanceTile berfungsi untuk memicu pengambilan kartu kesempatan oleh pemain.

Method:

- drawCard(player: Player): Card : mengambil dan mengembalikan kartu dari cardPool untuk pemain

Relasi:

- Menggunakan List<Card> sebagai kumpulan kartu
- Berinteraksi dengan Player

2.14. Kelas FestivalTile extends ActionTile

Kelas FestivalTile berguna untuk mengaktifkan efek festival pada properti milik pemain.

Method:

- applyFestivalEffect(player, property) : menerapkan efek festival pada properti milik pemain
- updateDuration(): memperbarui sisa durasi efek festival

Relasi:

- Berinteraksi dengan Player dan Property

2.15. Kelas TaxTile extends ActionTile

Kelas TaxTile digunakan untuk menghitung dan memungut pajak dari pemain sesuai jenis pajak yang berlaku.

Method:

- calculateTax(player): menghitung besar pajak yang harus dibayar pemain
- collectTax(player): memungut pajak dari pemain

Relasi:

- Menggunakan TaxType
- Berinteraksi dengan Player

2.16. Kelas SpecialTile extends ActionTile

Kelas SpecialTile berfungsi untuk menjalankan efek khusus sesuai jenis petak spesial yang diaktifkan.

Method:

- executeSpecial(player): menjalankan efek khusus pada pemain

Relasi:

- Menggunakan SpecialType
- Berinteraksi dengan Player

2.17. Kelas GoTile extends SpecialTile

Kelas GoTile berfungsi untuk memberikan gaji kepada pemain saat melewati atau mendarat di petak GO.

Method:

- giveSalary(player): menambahkan gaji kepada pemain

Relasi:

- Berinteraksi dengan Player

2.18. Kelas JailTile extends SpecialTile

Kelas JailTile berfungsi untuk mengatur kondisi pemain yang berada di penjara.

Method:

- sendToJail(player): memindahkan pemain ke penjara
- visitJail(player): menangani kondisi pemain yang hanya berkunjung
- attemptEscape(player): mencoba keluar dari penjara
- payFine(player): membayar denda untuk keluar dari penjara

Relasi:

- Berinteraksi dengan Player

2.19. Kelas FreeParkingTile extends SpecialTile

Kelas FreeParkingTile berfungsi untuk memberikan kondisi istirahat tanpa efek kepada pemain.

Method:

- rest(player): tidak melakukan aksi apapun terhadap pemain

2.20. Kelas GoToJailTile extends SpecialTile

Kelas GoToJailTile berfungsi untuk memindahkan pemain langsung ke penjara.

Method:

- sendToJail(player): memindahkan pemain ke penjara.

Relasi:

- Berinteraksi dengan Player

2.21. Kelas FileParser

Tanggung jawab utama: Menyediakan method parsing teks yang dipakai oleh ConfigParser dan GameLoader.

Method tokenizeLine() memecah satu baris teks menjadi vector<String> berdasarkan whitespace.

Method trim() menghapus spasi di awal dan akhir string. Keduanya bersifat protected sehingga hanya bisa diakses oleh child class, bukan dari luar.

FileParser sendiri tidak bisa diinstansiasi karena tidak ada gunanya parser yang tidak tahu file apa yang dibaca. Kelas ini untuk menghindari duplikasi kode. Tanpa FileParser, tokenizeLine() dan trim() harus ditulis ulang di ConfigParser dan GameLoader secara identik.

2.22. Kelas ConfigParser extends FileParser

Tanggung jawab utama: Membaca seluruh file konfigurasi dan membangun papan permainan dari nol saat New Game.

Atribut basePath menyimpan path ke direktori yang berisi file-file config.

loadConfig() adalah satu-satunya method public yang dipanggil oleh GameController.startGame() untuk menginisialisasi GameBoard. Di dalamnya, loadConfig() memanggil enam method private secara berurutan.

parsePropertyConfig() membaca property.txt yang berisi data properti ID, kode, nama, jenis, warna, harga, nilai gadai, harga upgrade, dan tabel sewa. Untuk setiap baris, method ini membuat objek Street, Railroad, atau Utility sesuai kolom JENIS, mengisi atribut-atributnya dari data yang diparsing, lalu push ke GameBoard.tiles.

parseRailroadConfig() membaca railroad.txt yang berisi mapping jumlah railroad ke biaya sewa, lalu mengisi rentTable di setiap objek Railroad yang sudah dibuat. **parseUtilityConfig()** membaca utility.txt yang berisi mapping jumlah utility ke faktor pengali, lalu mengisi multiplierTable di setiap objek Utility.

parseTaxConfig() membaca tax.txt yang berisi tiga nilai PPH flat, PPH persentase, dan PBM flat dan lalu mengisi flatAmount dan percentageRate di objek TaxTile yang sesuai.

parseSpecialConfig() membaca special.txt yang berisi gaji GO dan denda penjara, lalu mengisi GoTile.salary dan JailTile.fineAmount.

parseMiscConfig() membaca misc.txt yang berisi max turn dan saldo awal, lalu mengisi maxTurn di GameController (via WinConditionChecker.setMaxTurn()) dan startingBalance untuk inisialisasi uang setiap Player.

parseActionConfig() membaca aksi.txt dan membuat petak khusus berdasarkan kolom JENIS_PETAK dan kode petak: GoTile, JailTile, FreeParkingTile, GoToJailTile untuk SPESIAL; ChanceTile/CommunityChestTile untuk KARTU; TaxTile (memakai pphFlat/pphPercentage atau pbmFlat) untuk PAJAK; FestivalTile untuk FESTIVAL.

2.23. Kelas GameLoader extends FileParser

Tanggung jawab utama: Membaca file save dan restore seluruh state permainan yang tersimpan ke objek-objek game yang sudah ada.

GameLoader selalu dipanggil setelah ConfigParser karena file save tidak menyimpan data config (tabel sewa, harga, pajak), melainkan hanya state runtime (siapa pemilik properti, berapa uang pemain, posisi bidak, dll.). Jadi papan harus sudah terbangun dengan data config sebelum save file bisa diterapkan di atasnya.

validate() adalah method yang dipanggil sebelum **loadSave()** untuk memverifikasi apakah file save memiliki format yang benar. Jika file tidak ditemukan, kosong, atau strukturnya tidak sesuai spesifikasi, method ini melempar `FormatException` yang kemudian ditangkap oleh GameController untuk menampilkan pesan "File rusak atau format tidak dikenali."

loadSave() menerima nama file, GameBoard, dan TransactionLogger. Di dalamnya, method ini pertama-tama membaca baris pertama untuk mendapatkan turn saat ini dan max turn, lalu memanggil method-method private secara berurutan sesuai struktur file save.

readPlayerStates() membaca state setiap pemain. Username, uang, posisi, dan status. Set nilai-nilai tersebut ke objek Player yang sesuai di GameBoard.players.

readTurnOrder() membaca urutan giliran dan giliran aktif saat ini, lalu set `currentPlayerIndex` di GameBoard.

readPropertyStates() membaca state setiap properti seperti kode, jenis, pemilik, status, festival multiplier/duration, dan jumlah bangunan. Lalu set nilai-nilai tersebut ke objek Property yang sesuai di GameBoard.tiles.

readDeckState() membaca kartu-kartu yang tersisa di deck kemampuan.

readLogState() membaca entri-entri log yang tersimpan, lalu memasukkannya ke TransactionLogger lewat method `log()`.

2.24. Kelas GameSaver

Tanggung jawab utama: Menulis seluruh state permainan ke file .txt sesuai format yang didefinisikan spesifikasi.

Method **save()** Di dalamnya, `save()` pertama-tama menulis baris header (turn saat ini dan max turn) langsung karena hanya dua angka, lalu memanggil method-method private secara berurutan.

writePlayerStates() mengiterasi GameBoard.players dan untuk setiap pemain menulis username, uang, posisi, status, jumlah kartu, dan detail setiap kartu tangan (jenis, value, durasi) sesuai format state pemain di spesifikasi.

writeTurnOrder() menulis urutan giliran sebagai baris username yang dipisah spasi, diikuti baris giliran aktif saat ini.

writePropertyStates() mengiterasi tile-tile di GameBoard, memfilter yang merupakan Property, dan untuk setiap properti menulis kode, jenis, pemilik, status, festival multiplier/duration, dan jumlah bangunan.

writeDeckState() menulis kartu-kartu yang tersisa di deck. **writeLogState()** menulis semua string log dari TransactionLogger.getAll().

Jika file gagal ditulis (direktori tidak bisa ditulis, disk penuh, dsb.), GameSaver melempar FileNotFoundException yang ditangkap oleh GameController untuk menampilkan pesan "Gagal menyimpan file!".

2.25. Kelas TransactionLogger

Tanggung jawab utama: Mencatat setiap kejadian signifikan selama permainan sebagai formatted string dan menyediakan data log untuk tampilan dan save file.

Atribut **logs** adalah vector<String> yang menyimpan seluruh log selama sesi permainan. Setiap kali terjadi aksi di permainan seperti lempar dadu, beli properti, bayar sewa, bayar pajak, bangun rumah, gadai, tebus, pakai kartu, lelang, festival, bangkrut, save, load GameController memanggil log() dengan empat parameter: nomor turn, username pemain, jenis aksi, dan detail aksi.

formatEntry() mengubah keempat parameter tersebut menjadi satu string terformat, misalnya " 13 Uname1 DADU x` Lempar: 3+4=7 mendarat di Bandung (BDG)". String ini disimpan di vector logs.

getAll() mengembalikan seluruh isi logs dipakai oleh UI saat perintah CETAK_LOG tanpa argumen, dan oleh GameSaver.writeLogState() saat menyimpan game.

getRecent(n) mengembalikan n string terakhir dari logs dipakai oleh UI layer saat perintah CETAK_LOG N.

getCount() mengembalikan jumlah total entri.

clear() mengosongkan logs dipakai saat memulai game baru.

GameLoader.readLogState() membaca log dari file save dan memasukkannya kembali ke TransactionLogger, sehingga log dari sesi sebelumnya tetap tersedia setelah load.

2.26. Kelas WinConditionChecker

Tanggung jawab utama: Mengevaluasi apakah permainan sudah selesai dan menentukan pemenang berdasarkan aturan spesifikasi.

Atribut **maxTurn** menyimpan batas giliran maksimum yang dibaca dari misc.txt.

setMaxTurn() dipanggil oleh ConfigParser saat parsing config.

isMaxTurnValid() mengecek apakah maxTurn ≥ 1 jika tidak valid (maxTurn < 1), permainan berjalan tanpa batas turn dalam mode bankruptcy-only.

GameController.checkGameEnd() mendelegasi ke isGameOver() yang menggabungkan dua pengecekan.

isBankruptcyEnd() return true jika activePlayerCount == 1, artinya semua pemain kecuali satu sudah bangkrut.

isMaxTurnEnd() return true jika currentTurn sudah melewati maxTurn dan maxTurn valid.

isGameOver() return true jika salah satu dari keduanya terpenuhi.

Untuk mode BANKRUPTCY, pemenangnya cuma satu (pemain terakhir yang aktif). Untuk mode MAX_TURN, tiga comparator dijalankan berurutan:

compareByMoney() membandingkan uang tunai,

compareByPropertyCount() membandingkan jumlah properti yang dimiliki, dan

compareByCardCount() membandingkan jumlah kartu di tangan.

determineWinners() mengembalikan vector<Player*> berisi semua pemain yang seri di puncak setelah ketiga comparator dijalankan, sehingga UI bisa menampilkan para pemenang.

determineWinner() mengembalikan elemen pertama dari list tersebut.

Jika setelah ketiga perbandingan masih seri, semua pemain yang seri dianggap menang bersama.

2.27. Kelas `FileWriteException`

Tanggung jawab utama: Custom exception yang dilempar saat operasi penulisan file gagal.

Atribut **message** menyimpan deskripsi error, misalnya "Gagal menulis ke game_sesi1.txt: Permission denied".

Method **what()** mengembalikan message tersebut. Dilempar oleh `GameSaver.save()` saat ofstream gagal membuka atau menulis file. Ditangkap oleh `GameController` yang menampilkan "Gagal menyimpan file! Pastikan direktori dapat ditulis." melalui UI layer.

2.28. Kelas `FileFormatException`

Tanggung jawab utama: Custom exception yang dilempar saat file config atau file save memiliki format yang tidak sesuai atau tidak ditemukan.

Atribut **message** menyimpan deskripsi error spesifik, misalnya "property.txt tidak ditemukan" atau "Format save file tidak dikenali pada baris 5".

Method **what()** mengembalikan message tersebut. Dilempar oleh `ConfigParser` saat file config rusak dan oleh `GameLoader.loadSave()` saat file save rusak. Ditangkap oleh `GameController` yang meneruskan pesan error ke UI layer untuk ditampilkan.

2.29. Kelas `Player`

Tanggung jawab utama: Merepresentasikan satu pemain dalam permainan, menyimpan seluruh state personal pemain, uang, posisi di papan, status (ACTIVE/BANKRUPT/JAILED), kartu di tangan, dan flag per giliran.

operator+= digunakan untuk setiap penerimaan uang (gaji GO, sewa dari pemain lain, hasil lelang).

operator-= digunakan untuk semua tagihan paksa (sewa, pajak, sanksi kartu). Jika `shieldActive == true`, tagihan diblok tanpa menghabiskan shield.

payVoluntary() dipisahkan dari `operator-=` untuk pembayaran sukarela (beli properti, tebus gadai, bangun rumah, bid lelang). Pembayaran sukarela tidak kena shield.

calculateTotalWealth() menerima pointer ke `GameBoard` untuk menghitung total kekayaan. Jika board tidak diberikan, hanya uang tunai yang dikembalikan sebagai fallback.

receiveCard() mengembalikan bool yang bernilai true jika tangan pemain melebihi batas maksimal setelah kartu masuk.

resetTurnFlags() dipanggil oleh TurnManager di awal setiap giliran pemain. Method ini me-reset hasRolledThisTurn, hasUsedCardThisTurn, shieldActive, dan discountPercent sekaligus.

operator> dan operator< membandingkan uang tunai pemain. Digunakan oleh WinConditionChecker untuk menentukan peringkat pemenang saat kondisi MAX_TURN tercapai.

2.30. Kelas Dice

Tanggung jawab utama: Merepresentasikan dua dadu bersisi 6 dengan dukungan mode random dan mode manual untuk keperluan perintah ATUR_DADU.

rollRandom() menghasilkan nilai acak 1-6 untuk masing-masing dadu menggunakan rand() dengan seed sekali di konstruktor.

setManual() menerima dua nilai integer dan melempar std::invalid_argument jika salah satu nilai di luar rentang 1-6. CommandProcessor menangkap exception ini untuk menampilkan pesan error yang sesuai kepada user tanpa crash.

isDouble() mengembalikan true jika kedua nilai dadu sama. Digunakan oleh MovementHandler dan GameController untuk menentukan giliran tambahan dan deteksi triple double.

2.31. Kelas ActionCard

Tanggung jawab utama: Mendefinisikan kontrak bersama untuk semua kartu yang diambil dari petak aksi, Kesempatan dan Dana Umum.

ActionCard langsung diambil dari deck dan langsung dieksekusi tanpa disimpan. Karena itu, satu-satunya atribut adalah description yang menyimpan teks instruksi kartu sesuai spesifikasi.

Method **execute(player, context)** dijadikan abstract agar setiap subclass wajib mengimplementasikan efeknya sendiri, sementara **ChanceTile** dan **CommunityChestTile** bisa memanggil card->execute(player, context) lewat pointer base class tanpa perlu tahu jenis kartunya.

Parameter GameContext digunakan karena beberapa efek kartu memerlukan akses di luar pemain itu sendiri, misalnya kartu "Masuk Penjara" perlu MovementHandler.sendToJail(), dan kartu "Mundur 3 petak" perlu MovementHandler.movePlayer().

Relasi ke CardDeck<ActionCard> terjadi secara tidak langsung. CardDeck menyimpan objek-objek ActionCard, dan ChanceTile/CommunityChestTile memanggil deck.drawTop() untuk mengambil kartu teratas, yang hasilnya adalah pointer ke ActionCard yang kemudian di-execute().

2.32. Kelas ChanceCard

Tanggung jawab utama: Mengimplementasikan tiga efek kartu Kesempatan

execute() di ChanceCard mengimplementasikan salah satu dari tiga kemungkinan efek:

1. pergi ke stasiun terdekat,
2. mundur 3 petak,
3. masuk penjara.

Setelah execute() selesai, kartu dikembalikan ke discardPile di dalam CardDeck oleh ChanceTile dengan memanggil deck.discard(card). Ketika deck habis, CardDeck.reshuffleDiscardIntoDeck() otomatis dipanggil, sesuai mekanisme spesifikasi.

2.33. Kelas CommunityCard

Tanggung jawab utama: Mengimplementasikan tiga efek kartu Dana Umum

Strukturnya identik dengan ChanceCard, perbedaannya ada di efek yang diimplementasikan di execute():

1. ulang tahun (terima M100 dari setiap pemain),
2. biaya dokter (bayar M700),
3. mau nyaleg (bayar M200 ke setiap pemain).

Dua dari tiga efek ini melibatkan transaksi ke atau dari semua pemain lain, bukan hanya pemain yang mendarat. Karena itu `context.allPlayers` diakses di dalam `execute()` untuk iterasi semua pemain aktif dan melakukan transfer uang menggunakan operator `+=` dan `-=` milik `Player`.

Untuk efek pembayaran ke bank (biaya dokter), `execute()` cukup memanggil `player -= 700`. Jika pemain tidak mampu bayar, dicek lewat `player.getMoney() < amount` sebelum eksekusi. kondisi ini memicu alur kebangkrutan yang ditangani `GameController`. `CommunityCard` hanya bertanggung jawab mengeksekusi efek kartunya.

2.34. Kelas Skillcard & Turunannya

Tanggung jawab utama: Merepresentasikan kartu kemampuan spesial yang dapat diaktifkan pemain untuk memberikan efek taktis.

SkillCard adalah abstract class yang mendefinisikan kontrak bersama dimana semua kartu pasti punya **cardType**, **value**, **remainingDuration**, dan **method activate()** serta **getDescription()**. Dengan ini, `SkillCardManager` dan `Player` bisa memperlakukan semua kartu secara seragam lewat pointer ke base class, tanpa perlu tahu jenis kartunya ini adalah penggunaan polymorphism.

Method **decrementDuration()** di base class dipanggil oleh `SkillCardManager` setiap akhir giliran untuk semua kartu di tangan pemain, sehingga efek `DiscountCard` yang berlaku satu giliran bisa habis secara otomatis. Masing-masing turunan mengimplementasikan **activate()** dengan cara berbeda:

MoveCard memanggil `MovementHandler.movePlayer()` dengan nilai langkah yang sudah ditentukan acak saat kartu digenerate dan disimpan di `value`.

DiscountCard memanggil `player.setDiscount(value)` untuk mengaktifkan efek diskon. Method `applyDiscount()` dipanggil oleh Bank atau sistem lelang di domain Mekanisme saat menghitung harga beli.

ShieldCard memanggil `player.activateShield()`. Efeknya kemudian dicek oleh tile-tile di domain Mekanisme via `player.isShieldActive()` sebelum mengenakan biaya apapun.

TeleportCard meminta input posisi target, lalu memanggil `MovementHandler.teleportPlayer()` yang berbeda dari `movePlayer()` karena tidak menghitung petak yang dilewati.

LassoCard mengakses `GameContext.allPlayers` untuk memilih target, lalu memanggil `MovementHandler.pullPlayer()` untuk memindahkan lawan ke posisi pemain saat ini.

DemolitionCard mengakses `GameContext.board` untuk menemukan properti milik lawan, lalu menghancurkan satu bangunan di atasnya dengan berinteraksi ke domain Mekanisme.

Semua turunan menerima `GameContext` di parameter `activate()` ini yang memungkinkan tiap kartu mengakses komponen yang dibutuhkan tanpa coupling langsung ke `GameController`.

2.35. Kelas `CardDeck<T>` (template)

Tanggung jawab utama: Menyediakan struktur deck kartu generik dengan mekanisme draw dan reshuffle.

Dengan generic parameter `T`, satu implementasi `CardDeck` dipakai untuk tiga jenis deck yang berbeda: **`CardDeck<SkillCard>`** di `SkillCardManager`, dan potensial **`CardDeck<ActionCard>`** untuk deck Kesempatan dan **Dana Umum** di domain Mekanisme untuk menghindari duplikasi kode implementasi deck.

`reshuffleDiscardIntoDeck()` dipanggil otomatis oleh **`drawTop()`** ketika deck kosong, sehingga caller tidak perlu mengelola kondisi ini sendiri.

2.36. Kelas `TurnManager`

Tanggung jawab utama: Mengelola urutan giliran pemain dan nomor turn permainan. Satu turn didefinisikan sebagai satu putaran penuh di mana semua pemain aktif bermain satu kali.

`advanceToNextPlayer()` mencari pemain aktif berikutnya via `findNextActiveIndex()` yang meng-skip pemain `BANKRUPT`. Turn number hanya di-increment ketika perpindahan terjadi wrap-around (index berikutnya lebih kecil atau sama dari index saat ini), memastikan turn number naik tepat setelah semua pemain aktif bermain satu kali dalam satu siklus.

`isBankruptcyMode()` mengembalikan `true` jika `maxTurn <= 0` sesuai spesifikasi yang menyatakan `MAX_TURN` tidak valid (`<1`) berarti permainan berjalan tanpa batas turn hingga tersisa satu pemain.

`setTurnOrder()` dan `setCurrentPlayerByUsername()` disediakan khusus untuk kebutuhan load game oleh `GameLoader` agar urutan giliran dan posisi giliran aktif dapat dipulihkan dari file save.

`resetTurnFlags()` mendelegasikan ke `Player::resetTurnFlags()` dan dipanggil oleh `GameController` di awal setiap giliran pemain.

2.37. Kelas `MovementHandler`

Tanggung jawab utama: Mengeksekusi semua perpindahan pemain di papan beserta seluruh side effect yang mengikutinya.

Semua aksi yang mengubah posisi pemain dilewatkan melalui kelas ini, bukan langsung ke `Player`, karena setiap perpindahan punya konsekuensi yang perlu ditangani secara terpusat.

`movePlayer()` adalah method inti yang dipakai untuk lemparan dadu normal. Di dalamnya, method ini memanggil **`handlePassGo()`** untuk mengecek apakah pemain melewati petak GO. Jika ya, `player += goSalary` dieksekusi sebelum posisi diperbarui. Setelah posisi diperbarui, `GameController` mengambil tile di posisi baru dan memanggil `tile.onLand(player)` untuk memicu efek petak.

`teleportPlayer()` dipakai oleh `TeleportCard` berbeda dari `movePlayer()` karena tidak memanggil `handlePassGo()`

`pullPlayer()` dipakai oleh `LassoCard` untuk memindahkan pemain lawan. Karena yang dipindahkan bukan pemain aktif, method ini hanya mengubah posisi target tanpa memicu efek giliran.

`sendToJail()` dipakai dari dua tempat, **`GoToJailTile.onLand()`** di domain Mekanisme, dan oleh **`MovementHandler`** sendiri ketika consecutiveDoubles mencapai tiga. Method ini mengeset posisi pemain ke petak Penjara dan mengubah status menjadi JAILED.

`handleJailTurn()` menangani tiga skenario keluar penjara sesuai spesifikasi bayar denda, dapat double, atau habis tiga giliran wajib bayar. Return type `JailResult` memberi tahu `GameController` apa yang terjadi sehingga output yang tepat bisa ditampilkan ke UI.

2.38. Kelas `SkillCardManager`

Tanggung jawab utama: Mengelola siklus hidup kartu kemampuan dari generasi, distribusi, penggunaan, hingga pembuangan.

distributeCardToAll() dipanggil di awal setiap giliran oleh GameController, sesuai spesifikasi yang menyatakan semua pemain mendapat satu kartu acak tiap giliran. Di dalamnya, **distributeCardTo()** dipanggil per pemain dan jika `player.getHand().size() == 3`, method ini otomatis memicu prompt DROP sebelum kartu baru masuk, sesuai aturan maksimal tiga kartu di tangan.

useCard() memvalidasi dua kondisi sebelum eksekusi `player.hasUsedCard()` harus false, dan `player.hasRolled()` harus false. Jika valid, method memanggil `card->activate(player, context)` lewat pointer base class, memanfaatkan polymorphism untuk menjalankan efek yang sesuai tanpa switch-case.

dropCard() memindahkan kartu dari tangan pemain ke discardPile di dalam skillDeck. Ketika deck habis, `CardDeck.reshuffleDiscardIntoDeck()` dipanggil otomatis.

generateCard() membuat objek kartu baru dengan nilai acak sesuai jenisnya, MoveCard mendapat langkah acak, DiscountCard mendapat persentase acak sesuai spesifikasi yang menyatakan nilai ditentukan saat kartu pertama kali didapatkan.

2.39. Kelas GameContext

Tanggung jawab utama: Menjadi wadah referensi yang dilempar ke SkillCard, ActionCard, dan Tiles agar bisa mengakses komponen yang dibutuhkan tanpa coupling langsung.

Tanpa **GameContext**, signature `activate()` di abstract class harus mencantumkan semua kemungkinan parameter dari semua fungsi. Hal ini tidak mungkin untuk kartu karena tiap kartu butuh hal berbeda. Dengan GameContext, signature tetap seragam `activate(player, context)` dan tiap turunan tinggal mengambil apa yang dibutuhkan dari context.

Board diakses oleh DemolitionCard dan TeleportCard. `movementHandler` diakses oleh MoveCard, TeleportCard, dan LassoCard. `allPlayers` diakses oleh LassoCard dan DemolitionCard untuk memilih target lawan. `turnManager` diakses jika kartu perlu tahu konteks giliran saat ini, dll.

2.40. Kelas GameController

GameController bertindak sebagai penghubung utama antara input, logic, dan output, serta memastikan seluruh alur permainan berjalan sesuai aturan tanpa mencampur logika permainan dengan tampilan.

3. Justifikasi Desain

Desain sistem yang dipilih dengan menggunakan pendekatan pemrograman berorientasi objek, di mana setiap kelas memiliki tanggung jawab yang jelas dan terpisah. Komponen utama seperti GameController, GameBoard, Tile, Player, dan berbagai kelas turunan dirancang agar saling berinteraksi melalui interface yang sederhana. Pendekatan ini mempermudah pengembangan fitur baru, seperti penambahan jenis tile atau kartu, tanpa perlu mengubah struktur utama sistem. Selain itu, penggunaan inheritance pada Tile dan Property memungkinkan setiap jenis petak memiliki perilaku spesifik namun tetap dapat diproses secara umum melalui polymorphism.

Perubahan desain dari milestone 1 terutama terjadi pada peningkatan fleksibilitas dan pemisahan tanggung jawab. Pada tahap awal, beberapa logika masih terpusat atau bergantung pada struktur statis, seperti ukuran papan dan pengolahan tile. Pada milestone 2, desain diperbaiki dengan memperkenalkan papan dinamis berbasis vector, penggunaan parser konfigurasi (ConfigParser) untuk membangun board, serta pemisahan komponen seperti MovementHandler, TurnManager, dan SkillCardManager. Perubahan ini membuat sistem lebih scalable dan tidak bergantung pada hardcoded, sehingga dapat menyesuaikan jumlah tile dari file konfigurasi.

Kelebihan dari diagram kelas yang digunakan adalah tingkat extensibility yang tinggi. Dengan adanya abstract base class seperti Tile, Property, dan SkillCard, sistem mudah dikembangkan tanpa memodifikasi kode lama. Selain itu, pemisahan antara logic (GameController, MovementHandler) dan view (BoardView, GameView) membuat kode lebih terstruktur dan mudah dipelihara. Namun, kekurangannya adalah meningkatnya kompleksitas relasi antar kelas, terutama karena banyaknya dependensi tidak langsung melalui GameContext atau pointer, yang dapat menyulitkan debugging dan pemahaman awal.

Kendala utama dalam pemilihan desain OOP adalah menentukan batas tanggung jawab tiap kelas agar tidak terjadi duplikasi logika. Contohnya, keputusan untuk memindahkan seluruh logika perpindahan ke MovementHandler dan bukan di Player memerlukan penyesuaian di banyak bagian sistem. Selain itu, penggunaan inheritance yang cukup dalam juga menuntut perancangan interface yang konsisten agar tidak menimbulkan konflik implementasi di subclass. Tantangan lain adalah menjaga keseimbangan antara desain yang fleksibel dan kompleksitas implementasi, terutama ketika mengintegrasikan banyak komponen seperti parser, controller, dan berbagai jenis tile dalam satu alur permainan.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

1. Tile → Action Tile, SpecialTile, Property

2. Action Tile → CommunityChestTile, ChanceTile, FestivalTile, TaxTile,
3. SpecialTile → FreeParkingTile, GoTile, GoToJailTile, JailTile
4. Property → Street, Utility, Railroad

Semua petak papan punya kesamaan dasar (posisi, nama, warna) tapi mereka mempunyai perilaku yang berbeda-beda. Dengan Inheritance Tile yang memiliki fungsi OnLand(Perilaku papan), MovementHandler cukup panggil tile->onLand() tanpa perlu tahu jenis petak, tanpa memboroskan if-else.

```
✓ class ActionTile : public Tile {
protected:
    ActionType actionType;
```

```
✓ class ChanceTile : public ActionTile {
private:
    CardDeck<ActionCard>* deck;
```

```
class CommunityChestTile : public ActionTile {
private:
    CardDeck<ActionCard>* deck;
```

```
class FestivalTile : public ActionTile {
public:
```



```
class TaxTile : public ActionTile {  
private:
```

```
class Property : public Tile {
```

```
class Street : public Property
```

```
class Utility : public Property {
```

```
class Railroad : public Property
```

5. SkillCard → MoveCard, ShieldCard, DiscountCard, TeleportCard, LassoCard, DemolitionCard

4.2. Method/Operator Overloading

```
// Terima uang
Player& Player::operator+=(int amt) {
    if (amt > 0) money += amt;
    return *this;
}

// Pembayaran paksa (sewa, pajak, denda, efek kartu negatif)
Player& Player::operator-=(int amt) {
    if (amt <= 0) return *this;
}
```

Operator overloading digunakan pada kelas Player untuk mempermudah pengelolaan uang dan perbandingan antar pemain. operator+= digunakan ketika pemain menerima uang, sedangkan operator-= digunakan ketika pemain mengeluarkan uang. Selain itu, operator> dan operator< digunakan untuk membandingkan kondisi pemain berdasarkan jumlah uang yang dimiliki.

4.3. Template & Generic Classes

```
class CardDeck {
private:
    std::vector<T> deck;
    std::vector<T> discardPile;
```

Template yang digunakan adalah CardDeck<T> dan container STL seperti vector<T>. CardDeck<T> digunakan agar sistem deck kartu dapat dipakai untuk berbagai jenis kartu tanpa membuat ulang class deck yang berbeda. Selain itu, vector<T> digunakan untuk menyimpan kumpulan data seperti pemain, kartu, tile, dan log.

4.4. Exception

```
class FileFormatException {
private:
    std::string message;

public:
    explicit FileFormatException(const std::string& msg);
    const char* what() const;
}
```

Exception digunakan pada FileFormatException untuk menangani kesalahan pada proses pembacaan file konfigurasi dan file save. FileFormatException digunakan ketika terjadi kesalahan seperti format file tidak sesuai atau data tidak valid. Dengan exception, kesalahan dapat langsung dihentikan dan ditangani tanpa mengganggu alur utama program, sehingga kode menjadi lebih rapi dan mudah di-maintain.

4.5. C++ Standard Template Library

```
const std::vector<std::shared_ptr<Player>>& getPlayers() const;
std::shared_ptr<Player> getPlayerByUsername(const std::string& username);
```

```
class GameBoard {
private:
    std::vector<std::unique_ptr<Tile>> tiles;
    std::vector<std::shared_ptr<Player>> players;
```

C++ Standard Template Library (STL) yang digunakan adalah std::vector, yang berfungsi untuk menyimpan koleksi objek secara dinamis, seperti daftar tile dan pemain pada GameBoard. Penggunaan vector memungkinkan jumlah elemen menyesuaikan isi config tanpa perlu alokasi manual, sehingga sangat sesuai untuk implementasi papan dinamis. Selain itu, digunakan juga std::shared_ptr untuk menyimpan objek Player, karena objek ini diakses oleh banyak komponen seperti GameController, BoardView, dan GameBoard. Dengan shared_ptr, pengelolaan memori dilakukan secara otomatis melalui reference counting, sehingga mengurangi risiko memory leak atau dangling pointer.

4.6. Konsep OOP lain

1. Abstract Base Class

Tile, ActionTile, SpecialTile, Property, dan SkillCard dibuat menjadi ABC agar memastikan semua subclass mengimplementasikan method wajib dan mengikuti prinsip Open - Closed Principle

2. Aggregation dan Composition

GameController dibuat memiliki relasi Aggregation/Composition dengan GameBoard, GameSaver, GameLoader, dan TransactionLogger agar setiap bagian sistem memiliki tanggung jawab yang terpisah. Dengan relasi ini, GameController cukup mengoordinasikan alur permainan, sedangkan penyimpanan data, pemuatan data, papan permainan, dan pencatatan log dikelola oleh kelas masing-masing.

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. Papan Dinamis

Class GameBoard memiliki attribute tiles yang berupa vector dengan tipe unique_ptr of Tile

```
class GameBoard {  
private:  
    std::vector<std::unique_ptr<Tile>> tiles;
```

Class ConfigParser (Ojan tolong jelasin pipeline configparser buat ngeload board)

Pendekatan papan dinamis dilakukan dengan mengganti seluruh logika berbasis indeks statis menjadi perhitungan berbasis board.getTileCount(). Papan dibagi menjadi empat sisi, dan posisi tile dihitung secara matematis sehingga sistem dapat mendukung ukuran papan yang fleksibel. Perubahan jumlah tile akan memengaruhi dimensi visual papan, baik dari segi lebar maupun tinggi.

5.2. Bonus Kreasi Mandiri

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
CommandProcessor	13524129	13524129
BoardView	13524129	13524129
PropertyView	13524129	13524129
GameView	13524129	13524129
PlayerView	13524129	13524129
Tile	13524121	13524121
Property	13524121	13524121
Street	13524121	13524121
Railroad	13524121	13524121
Utility	13524121	13524121
ActionTile	13524121	13524121
ChanceTile	13524121	13524121
CommunityChestTile	13524121	13524121
FestivalTile	13524121	13524121
TaxTile	13524121	13524121
SpecialTile	13524121	13524121
GoTile	13524121	13524121

JailTile	13524121	13524121
FreeParkingTile	13524121	13524121
GoToJailTile	13524121	13424121
FileParser	13524141	13524141
ConfigParser	13524141	13524141
GameLoader	13524141	13524141
GameSaver	13524141	13524141
TransactionLogger	13524141	13524141
WinConditionChecker	13524141	13524141
FileWriteException	13524141	13524141
FileFormatException	13524141	13524141
Player	13524125	13524125
Dice	13524125	13524125
ActionCard	13524125	13524125
ChanceCard	13524125	13524125
CommunityCard	13524125	13524125
Skillcard	13524125	13524125
CardDeck<T>	13524125	13524125

TurnManager	13524125	13524125
MovementHandler	13524125	13524125
SkillCardManager	13524125	13524125
GameContext	13524125	13524125
GameController	13524121, 13524125, 13524129, 13524141	13524121, 13524125, 13524129, 13524141

Lampiran

Lampiran A. Form Asistensi

Link Drive: https://drive.google.com/drive/u/3/folders/1Qh8-YIKYA15qq9QIPRtba_UxrBsd9Oj1